



Servicio público de Salud Red Norte

Plataforma inteligente para la gestión de listas de espera hospitalarias

Equipo

Antonella Cuvertino - Miriam Hammami

Marcelo Crisóstomo Carrasco

Contexto

El Servicio de Salud RedNorte maneja una gran cantidad de pacientes en distintos hospitales y centros médicos. Actualmente, uno de los principales problemas es la gestión de las listas de espera, ya que existen sistemas separados que no se comunican bien entre sí.

Esto genera situaciones como:

- Permanencia excesiva de pacientes en listas de espera por falta de priorización eficiente.
- Pérdida crítica de horas médicas debido a cancelaciones de citas de último momento sin respaldo de contingencia.
- Inexistencia de herramientas automatizadas para la reasignación dinámica de citas liberadas en la red.
- Escasa transparencia y visibilidad hacia los pacientes respecto al estado real de sus solicitudes de atención.
- Dificultad técnica para centralizar datos y generar analítica de indicadores clave de gestión hospitalaria.

En base a esto, surge la necesidad de una plataforma que centre la información y haga más eficiente todo el proceso.

Índice

Resumen	2
Índice	3
Introducción	4
Solución Propuesta	5
Tecnologías a Utilizar	6
Justificación	7
Conclusiones	8

Introducción

El Servicio de Salud RedNorte administra una red de hospitales, centros de atención primaria y clínicas especializadas, los cuales atienden diariamente a un alto volumen de pacientes en distintas áreas médicas. Esta realidad implica manejar una gran cantidad de solicitudes de atención, incluyendo consultas, procedimientos y cirugías.

En este contexto, uno de los principales desafíos del sistema es la gestión de las listas de espera, ya que la demanda por servicios de salud supera en muchos casos la capacidad disponible. Esto genera demoras que afectan directamente la calidad de atención y la experiencia de los pacientes.

Actualmente, cada establecimiento de salud utiliza sistemas propios para registrar y gestionar la información, lo que provoca una falta de integración entre ellos. Como consecuencia, no existe una visión unificada del estado de los pacientes dentro de la red, dificultando la coordinación y toma de decisiones.

2. Identificación de Problemas

A partir del análisis del contexto, se identifican una serie de problemáticas que afectan tanto la eficiencia operativa como la calidad del servicio.

2.1 Falta de integración entre sistemas

Uno de los principales problemas es que los hospitales y centros de salud operan con sistemas independientes que no están completamente conectados entre sí. Esto genera fragmentación de la información, duplicidad de registros y dificultad para compartir datos relevantes.

Como resultado, un paciente puede estar en lista de espera en distintos centros sin que exista una coordinación adecuada, lo que reduce la eficiencia del sistema en su conjunto.

2.2 Largos tiempos de espera

Debido a la alta demanda y a la falta de optimización en la asignación de recursos, muchos pacientes permanecen largos periodos en lista de espera. Este problema no siempre se debe a la falta de horas médicas, sino también a una gestión ineficiente de las mismas.

Esto impacta directamente en la percepción del servicio y puede incluso agravar las condiciones de salud de los pacientes.

2.3 Pérdida de horas médicas por cancelaciones

Otro problema relevante es la cancelación de citas médicas. Cuando un paciente cancela una hora, muchas veces esta no es reasignada a tiempo, lo que genera una pérdida de recursos disponibles.

Esta situación evidencia la falta de herramientas automatizadas que permitan reaccionar rápidamente ante cambios en la agenda.

2.4 Escasa visibilidad para los pacientes

Actualmente, los pacientes no cuentan con mecanismos claros para conocer el estado de su solicitud o su posición en la lista de espera. Esto genera incertidumbre, aumenta la carga de consultas administrativas y afecta la confianza en el sistema.

2.5 Dificultad en el análisis de información

Finalmente, la falta de integración y centralización de datos dificulta la generación de indicadores relevantes, como tiempos promedio de espera, tasas de cancelación o eficiencia en el uso de horas médicas.

Esto limita la capacidad de la institución para tomar decisiones informadas y mejorar continuamente sus procesos.

3. Análisis de necesidades del sistema

A partir de las problemáticas identificadas, se pueden definir las principales necesidades que debe cubrir la solución tecnológica propuesta.

En primer lugar, se requiere una **centralización de la información**, que permita unificar los datos provenientes de distintos centros de salud y entregar una visión global del estado de los pacientes.

En segundo lugar, es necesario mejorar la **gestión de las listas de espera**, incorporando mecanismos que permitan priorizar pacientes y optimizar la asignación de citas médicas.

Además, el sistema debe incluir funcionalidades que permitan la **reasignación automática de horas médicas**, con el fin de reducir la pérdida de recursos ante cancelaciones.

Otro aspecto clave es mejorar la **comunicación con los pacientes**, entregando información clara y accesible sobre el estado de sus solicitudes.

Finalmente, se requiere que la solución permita la **generación de indicadores y análisis de datos**, apoyando la toma de decisiones a nivel institucional.

4. Enfoque de solución

Considerando todo lo anterior, la solución propuesta se orienta a desarrollar una plataforma digital basada en una arquitectura de microservicios.

Este enfoque permite dividir el sistema en componentes independientes, cada uno responsable de una funcionalidad específica, lo que facilita la escalabilidad, el mantenimiento y la evolución del sistema en el tiempo.

Sin embargo, es importante destacar que, para este proyecto, se prioriza una arquitectura simple y coherente, evitando complejidades innecesarias. Esto permite asegurar una implementación viable en etapas posteriores, manteniendo un equilibrio entre calidad técnica y factibilidad de desarrollo.

5. Propuesta de Arquitectura

A partir del análisis realizado, se propone una arquitectura basada en microservicios que permita abordar los problemas detectados de forma directa, manteniendo un diseño simple, modular y coherente con una futura implementación.

La propuesta se enfoca en resolver los puntos críticos identificados: falta de integración, pérdida de horas médicas, baja visibilidad para los pacientes y dificultad en la gestión de listas de espera.

En lugar de diseñar una arquitectura compleja, se opta por un conjunto reducido de microservicios bien definidos, cada uno con una responsabilidad clara.

5.1 Principios de diseño adoptados

Para asegurar coherencia entre el análisis y la solución, se definieron los siguientes principios:

- Separación de responsabilidades: cada microservicio cumple una función específica
- Bajo acoplamiento: los servicios funcionan de forma independiente
- Simplicidad: se evita sobrecargar la arquitectura con componentes innecesarios
- Escalabilidad progresiva: el sistema puede crecer sin necesidad de rediseño completo

Estos principios permiten construir una solución que no solo responde al problema actual, sino que también puede evolucionar en el tiempo.

5.2 Definición de microservicios

La selección de microservicios se realiza directamente en función de las necesidades detectadas en el análisis.

Servicio de Pacientes

Este servicio se encarga de gestionar la información básica de los pacientes, incluyendo datos personales y su identificación dentro del sistema.

Su existencia responde a la necesidad de centralizar la información, evitando duplicidad de registros entre distintos centros de salud.

Servicio de Lista de Espera (núcleo del sistema)

Este es el servicio principal de la solución. Administra el registro de pacientes en espera, su estado y su nivel de prioridad.

Permite organizar de manera estructurada las solicitudes de atención, resolviendo directamente el problema de desorden y falta de control en las listas.

Servicio de Citas

Gestiona la disponibilidad de horas médicas y la asignación de citas a los pacientes.

Este servicio permite tener un control claro de la agenda médica, facilitando la coordinación entre pacientes y profesionales.

Servicio de Reasignación

Este servicio aborda uno de los problemas más críticos detectados: la pérdida de horas médicas por cancelaciones.

Su función es identificar citas canceladas y reasignarlas automáticamente a otros pacientes en lista de espera, priorizando según criterios definidos.

Se plantea como un servicio simple, basado en reglas, evitando complejidad innecesaria en esta etapa.

API Gateway

El API Gateway actúa como punto de entrada único al sistema, recibiendo todas las solicitudes desde el frontend y dirigiéndose al microservicio correspondiente.

Su implementación permite simplificar la comunicación y centralizar aspectos como seguridad y control de acceso.

5.3 Comunicación entre servicios

La comunicación entre los microservicios se realizará mediante **peticiones HTTP (REST)**, lo que permite una implementación sencilla y ampliamente soportada.

Se opta por una comunicación síncrona en esta etapa, ya que:

Proyecto Red Norte sistema de salud - Miriam Hammami - Antonella Cuvertino

- Es más fácil de implementar y entender
- Permite un control directo de las solicitudes
- Es suficiente para los requerimientos actuales del sistema

Esta decisión se alinea con el enfoque de simplicidad definido previamente.

5.4 Persistencia de datos

A diferencia de arquitecturas más complejas, en esta propuesta se opta por una base de datos compartida entre los microservicios.

Esta decisión se toma considerando el contexto del proyecto, donde se prioriza la simplicidad, la coherencia y la facilidad de implementación por sobre una separación completa de la persistencia.

El uso de una base de datos centralizada permite:

- Reducir la complejidad en la gestión de datos
- Facilitar la implementación de los servicios
- Evitar problemas de sincronización entre múltiples bases de datos
- Mantener consistencia en la información

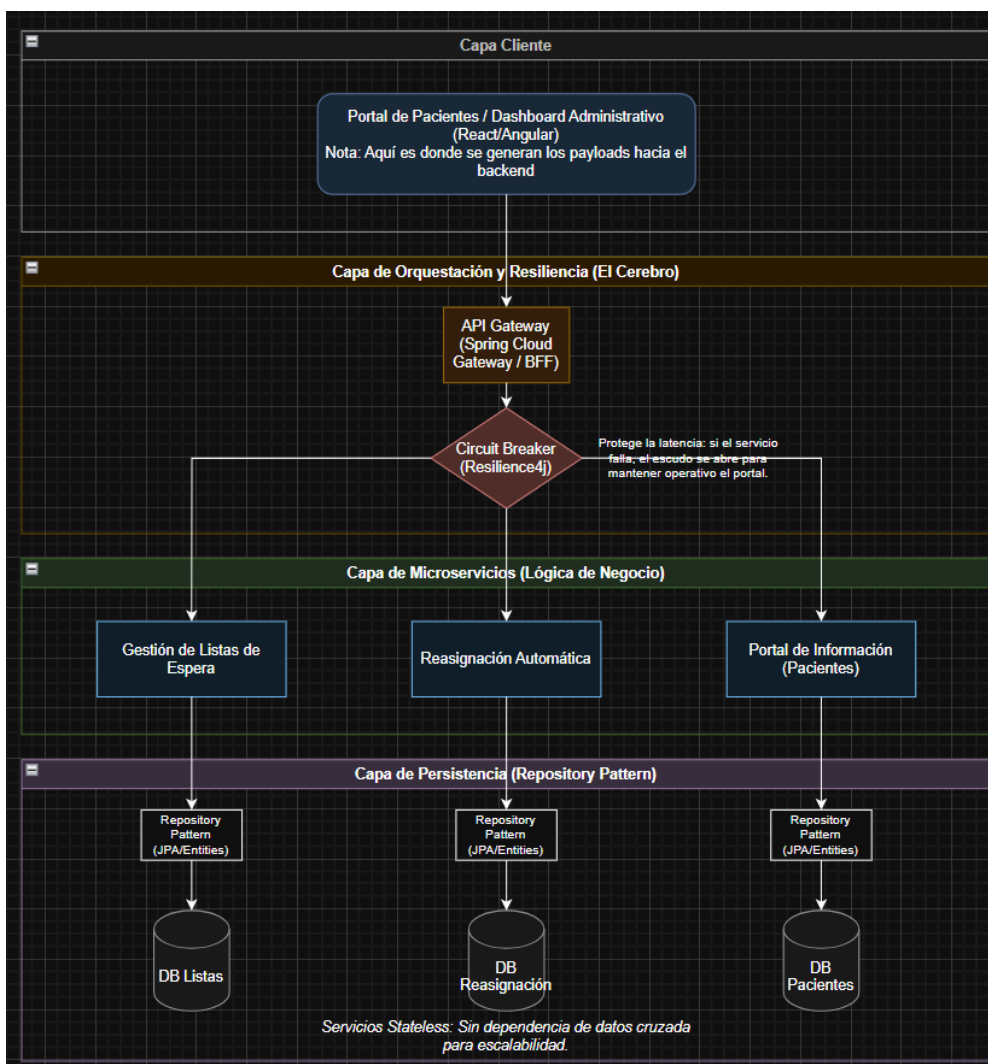
Si bien en entornos productivos de gran escala es común que cada microservicio tenga su propia base de datos, en este caso se considera que una base compartida es suficiente para cumplir con los requerimientos del sistema.

5.5 Justificación de la arquitectura propuesta

La arquitectura definida responde directamente a los problemas identificados en el análisis:

- La falta de integración se soluciona mediante un sistema centralizado con microservicios conectados
- La pérdida de horas médicas se aborda con el servicio de reasignación
- La gestión de listas se mejora con un servicio dedicado
- La organización del sistema se logra mediante la separación de responsabilidades

Además, se evita una sobreingeniería, lo que permite que la solución sea viable de implementar dentro del contexto académico.



5. Selección de Herramientas y Estrategias de Implementación

Con el fin de asegurar la eficiencia técnica y cumplir con los requerimientos de la Sección 2 del caso, se han seleccionado las siguientes herramientas:

Frameworks Modernos (React/Angular/Vue):		Se emplearán para desarrollar una interfaz intuitiva y responsiva, garantizando que los facultativos y pacientes tengan una experiencia de usuario (UX) óptima.
Backend For Frontend (BFF):		Estrategia que optimiza el consumo de datos desde el frontend, actuando como un intermediario que adapta la información de los microservicios a las necesidades específicas de la interfaz de usuario
Arquetipos Personalizados Maven		Herramienta clave para la gobernanza de la

		arquitectura, permitiendo estandarizar la creación de nuevos microservicios bajo una estructura técnica común y coherente.
Módulos Reutilizables:	NPM	Se utilizarán para empaquetar componentes de interfaz de usuario, asegurando una consistencia visual y funcional en toda la plataforma y acelerando los ciclos de desarrollo.
JPA y Entidades:		Para la persistencia de datos orientada a objetos, permitiendo una integración fluida con los microservicios independientes y facilitando el mantenimiento del modelo de datos.

Git & Git Flow		Estrategia de versionamiento esencial para el trabajo colaborativo, permitiendo una gestión de ramas que separa el desarrollo de nuevas funcionalidades de la estabilidad de la rama de producción.
---------------------------	--	--

Frameworks Modernos (React/Angular/Vue): Se emplearán para desarrollar una interfaz intuitiva y responsiva, garantizando que los facultativos y pacientes tengan una experiencia de usuario (UX) óptima.

Backend For Frontend (BFF): Estrategia que optimiza el consumo de datos desde el frontend, actuando como un intermediario que adapta la información de los microservicios a las necesidades específicas de la interfaz de usuario.

Arquetipos Maven Personalizados: Herramienta clave para la gobernanza de la arquitectura, permitiendo estandarizar la creación de nuevos microservicios bajo una estructura técnica común y coherente.

Módulos NPM Reutilizables: Se utilizarán para empaquetar componentes de interfaz de usuario, asegurando una consistencia visual y funcional en toda la plataforma y acelerando los ciclos de desarrollo.

JPA y Entidades: Para la persistencia de datos orientada a objetos, permitiendo una integración fluida con los microservicios independientes y facilitando el mantenimiento del modelo de datos.

Git & Git Flow: Estrategia de versionamiento esencial para el trabajo colaborativo, permitiendo una gestión de ramas que separa el desarrollo de nuevas funcionalidades de la estabilidad de la rama de producción.

Ética, Seguridad y Sostenibilidad en la Arquitectura

Seguridad y Privacidad

El diseño arquitectónico prioriza la protección de la ficha clínica electrónica. Mediante el uso del API Gateway, se implementan capas de autenticación y autorización robustas, asegurando que solo el personal clínico autorizado acceda a datos sensibles. El cumplimiento de estándares de privacidad es intrínseco al diseño, garantizando que el flujo de información entre hospitales cumpla con la normativa vigente de protección de datos personales en salud.

Sostenibilidad Técnica

La sostenibilidad se aborda a través de la reducción de la deuda técnica mediante la modularización extrema. El uso de patrones como *Repository* y microservicios independientes facilita que la plataforma sea mantenible a largo plazo, permitiendo actualizaciones tecnológicas parciales sin necesidad de reescribir el sistema completo. Esta flexibilidad asegura que la inversión tecnológica de RedNorte sea resiliente ante cambios en las políticas de salud pública o avances en el hardware.

Impacto en el Entorno

Desde la perspectiva de *Green IT*, la arquitectura promueve la optimización de recursos al reducir la subutilización de infraestructura crítica. Al minimizar las horas médicas perdidas, se optimiza no solo el tiempo del personal, sino también la energía y los insumos operativos de pabellones y oficinas que, de otro modo, quedarían ociosos. Este enfoque reduce el desperdicio operativo y maximiza el beneficio social por cada unidad de recurso invertido en la red.

6. Implementación de patrones de diseño y justificación

En la arquitectura propuesta se incorporan tres patrones de diseño específicos: Repository Pattern, Factory Method y Circuit Breaker. La selección de estos patrones responde a la necesidad de estructurar el sistema de manera ordenada, facilitar su implementación y asegurar su correcto funcionamiento en un entorno de microservicios.

A continuación, se describe cómo se aplicará cada patrón dentro de la solución propuesta.

6.1 Repository Pattern

El Repository Pattern se utilizará en cada microservicio para gestionar el acceso a la base de datos.

Dado que el sistema trabaja con una base de datos centralizada, cada servicio (Paciente, Lista de Espera, Citas y Reasignación) implementará su propio repositorio para acceder a las entidades que le corresponden, evitando que la lógica de negocio interactúe directamente con la base de datos.

Por ejemplo:

- El servicio de Pacientes tendrá un repositorio para gestionar los datos de los pacientes
- El servicio de Lista de Espera manejará su propio repositorio para las solicitudes
- El servicio de Citas accederá a la información de agenda médica

Esto permite mantener el código organizado y facilita modificaciones futuras en la persistencia sin afectar la lógica del sistema.

Su implementación es clave para mantener separación de responsabilidades y claridad en el desarrollo.

6.2 Factory Method

El patrón Factory Method se utilizará para la creación de objetos dentro del sistema, evitando instancias directas en la lógica de negocio.

En este proyecto, su aplicación se puede observar principalmente en la creación de citas médicas. Dependiendo del contexto, una cita puede tener distintas características (por ejemplo, tipo de atención o prioridad), por lo que se define un método encargado de generar estas instancias.

En lugar de crear los objetos directamente, el sistema delega esta responsabilidad a una fábrica, lo que permite encapsular la lógica de creación.

Esto facilita la extensión del sistema en el futuro, ya que se pueden agregar nuevos tipos de objetos sin modificar el código existente.

Su uso permite mantener flexibilidad sin aumentar la complejidad actual del sistema.

6.3 Circuit Breaker

El patrón Circuit Breaker se utilizará para manejar posibles fallos en la comunicación entre microservicios.

En la arquitectura propuesta, existen dependencias entre servicios. Por ejemplo, el servicio de Reasignación necesita consultar información desde el servicio de Lista de Espera y el servicio de Citas para poder reasignar horas médicas.

Si alguno de estos servicios no responde, el Circuit Breaker permite interrumpir temporalmente las solicitudes hacia ese servicio, evitando bloqueos o tiempos de espera prolongados.

Esto asegura que el sistema continúe funcionando incluso cuando uno de sus componentes presenta fallas.

Su implementación mejora la resiliencia del sistema y evita fallos en cascada.

6.4 Justificación de la selección de patrones

Los patrones seleccionados fueron elegidos en función de su utilidad dentro del contexto del sistema, evitando incorporar soluciones innecesarias o complejas.

- El Repository Pattern permite organizar el acceso a datos de forma clara y mantenible
- El Factory Method facilita la creación de objetos y la extensión del sistema
- El Circuit Breaker asegura estabilidad en la comunicación entre servicios

En conjunto, estos patrones permiten cumplir con los requerimientos técnicos del proyecto, asegurando que el sistema sea estructurado, escalable y preparado para futuras mejoras.

Falta el Diagrama

Las Pruebas postman junit

microservicio Pacientes

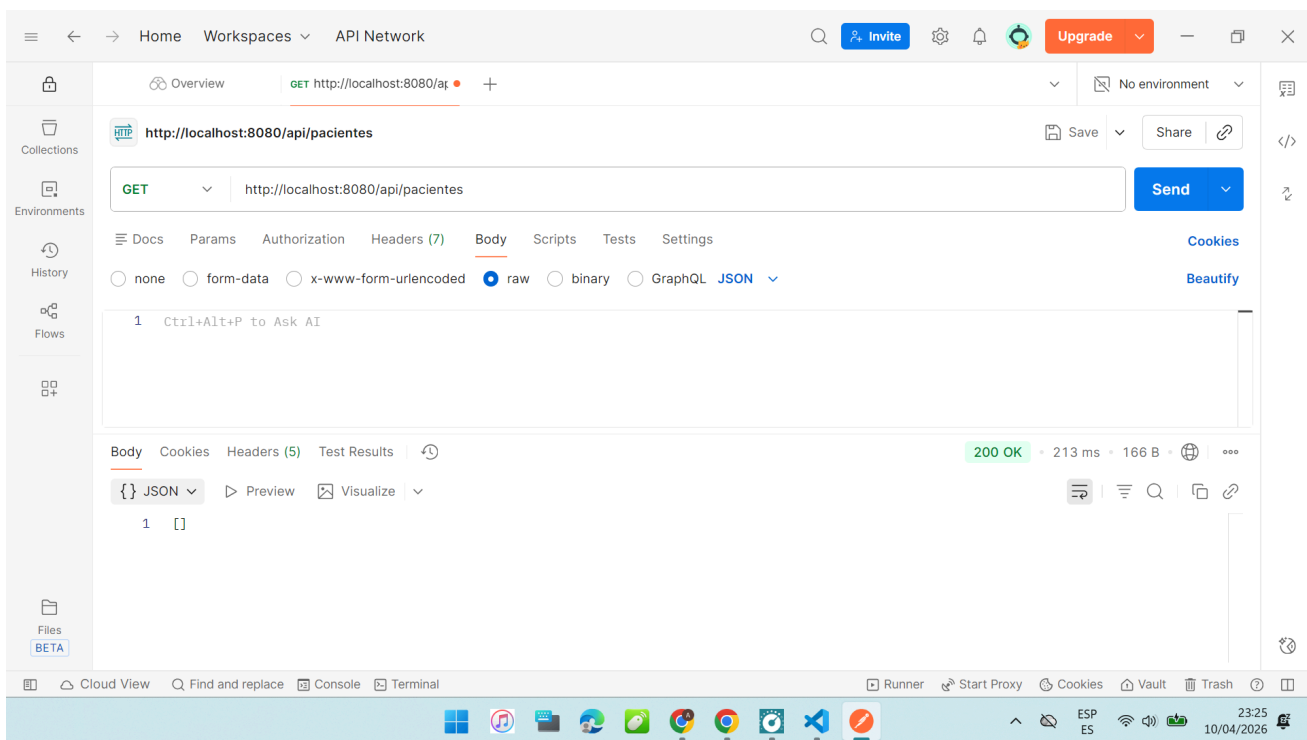
test corriendo e puerto 8080 utilizamos spring-boot:run para poder correr los test y poder generar los endpoints a utilizar.

```

PS C:\Users\anton\Desktop\servicio-pacientes> .\mvnw.cmd spring-boot:run
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.rednorte:servicio-pacientes >-----
[INFO] Building 0.0.1-SNAPSHOT
[INFO] from pom.xml
[INFO]
[INFO] -----[ jar ]-----
[INFO]
[INFO] >>> spring-boot:4.0.5:run (default-cli) > test-compile @ servicio-pacientes >>>
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ servicio-pacientes ---
[INFO] Copying 1 resource from src/main/resources to target/classes
[INFO] Copying 0 resource from src/main/resources to target/classes
[INFO]
[INFO] --- compiler:3.14.1:compile (default-compile) @ servicio-pacientes ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ servicio-pacientes ---
[INFO] skip non existing resourceDirectory C:\Users\anton\Desktop\servicio-pacientes\src\test\resources
[INFO]
[INFO] --- compiler:3.14.1:testCompile (default-testCompile) @ servicio-pacientes ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] <<< spring-boot:4.0.5:run (default-cli) < test-compile @ servicio-pacientes <<<
[INFO]
[INFO] --- spring-boot:4.0.5:run (default-cli) @ servicio-pacientes ---
[INFO] Attaching agents: []
  
```

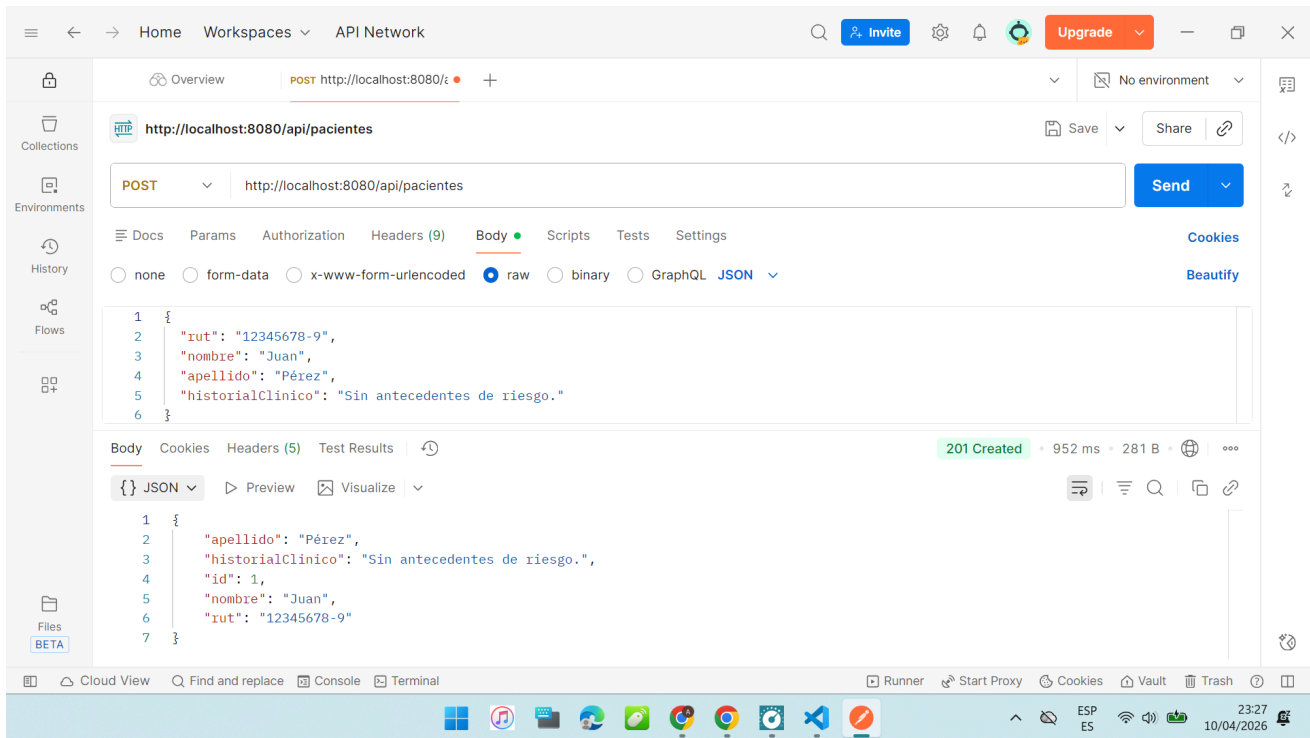
endpoint GET

Lista todos los pacientes registrados en el microservicio



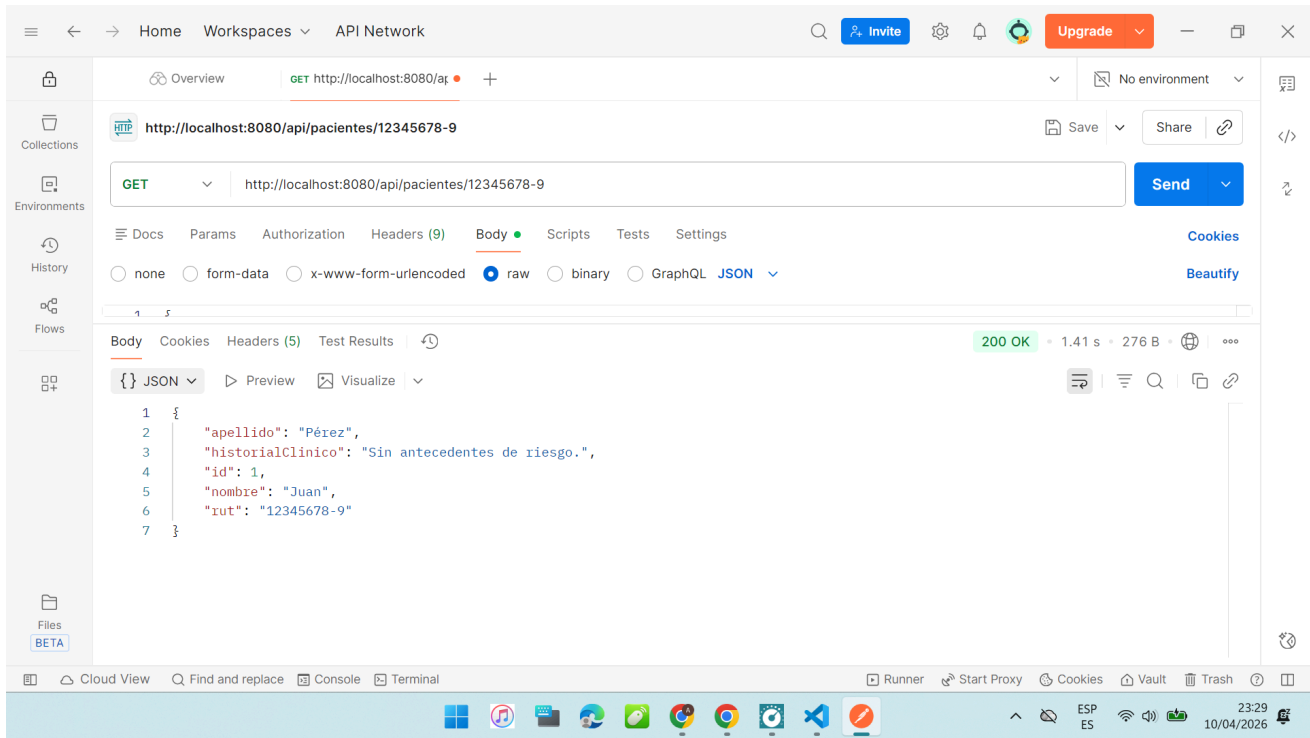
Proyecto Red Norte sistema de salud - Miriam Hammami - Antonella Cuvertino

Endpoint Post: Crea un paciente nuevo y lo almacena en la base de datos.



endpoint Get para buscar por rut

Busca un paciente por su RUT y devuelve sus datos.



Repositorio:

<https://github.com/Antonellacuvertino/Rednorte.git>